

CSCI 135 Assignment – Functions

CSCI 135 Assignment – Functions

October 25th / 27th, 2023

Due Date: Sunday, 10/29/2023 11:59PM

Points: 70

Topic: Functions

The goal of this assignment is to practice writing functions in Python. You will write a variety of functions. Continue making your code organized and readable. Pay attention to your header comments and include meaningful comments in-line with your code. Use proper indentation and understandable variable names. Use whitespace to help with readability and logical grouping of sections of code.

Writing reusable code and writing code for someone else is an application of functions.

Submission:

After following the instructions below, please submit the Python script in Moodle. Upload this as a single file – do not zip it. Be sure the file has a header section including the file name, your name, a description of the program, and other useful information such as example usage.

- GradingFunctions.py

Grading:

The program/script is worth 70 points. You will be graded according to the following criteria:

	Points
Header Comment for Entire Script	5
Descriptive Comments Throughout Code	5
Header Comments for Functions	5
printGradingScale() works and written correctly	10
getGradeByScore() works and written correctly	10
getGradeByScores() works and written correctly	10
getGradeByPoints() works and written correctly	10
getGradeRange() works and written correctly	10
Includes and uses cutoffs and letterGrades lists	5
Total	70

GradingFunctions.py

Write a script named GradingFunction.py that does the following:

For this assignment, you will write a script that includes five functions related to a grading scale and to computing grades according to the grading scale. This script is different from prior assignments in that what you submit will not run stand-alone. Instead, I will write code that calls your functions to test them. You should test your functions thoroughly, but you should not leave active test code in your submittal. (You can comment out test code so it doesn't get executed in the final version of your script.)

Include the following lists in your code immediately following your header comments, prior to all other code.

```
cutoffs = [100, 93, 90, 87, 83, 80, 77, 73, 70, 67, 63, 60, 0]
letterGrades = ['A', 'A-', 'B+', 'B', 'B-', 'C+', 'C', 'C-', 'D+', 'D', 'D-', 'F']
```

You must use these lists in your code that implements the following functions. Do not hard-code these values in your functions. Reference the lists.

Please implement the following functions in your script:

Function: printGradingScale()

Arguments: None

Returns: Nothing

Description:

Prints the grading scale to standard output using print().

Example use:

```
>>> printGradingScale()
Percent Range      Letter Grade
-----
 93 to 100%      A
 90 to <93%     A-
 87 to <90%     B+
 83 to <87%      B
 80 to <83%     B-
 77 to <80%     C+
 73 to <77%      C
 70 to <73%     C-
 67 to <70%     D+
 63 to <67%      D
 60 to <63%     D-
  Below 60%      F
```

Function: getGradeByScore(score)

Arguments:

score: a floating-point value between 0.0 and 1.0, inclusive, representing the percentage scored. For example, 0.9 represents 90%.

Returns:

A string containing the letter grade.

Description:

Returns the letter grade corresponding to the score. For example, 'A-' is returned for a score of 0.9.

Example use:

```
>>> strGrade = getGradeByScore(0.9)
>>> print(strGrade)
A-
```

Function: getGradeByScores(scores)

Arguments:

scores: a list of floating-point values between 0.0 and 1.0, inclusive, representing the percentage scored on individual assignments. For example, 0.9 represents 90%.

Returns:

A string containing the letter grade.

Description:

Returns the letter grade corresponding to the average of the scores. For example, 'B' is returned for a list of scores [0.95, 0.8, 0.85].

This function should call your getGradeByScore() function to get the final grade string, given the computed average of the scores.

Example use:

```
>>> strGrade = getGradeByScores([0.95, 0.8, 0.85])
>>> print(strGrade)
B
```

Function: getGradeByPoints(pointsEarned, pointsPossible)

Arguments:

pointsEarned: a list of non-negative integers representing the number of points earned on individual assignments. Items corresponds to those in the pointsPossible list.

pointsPossible: a list of non-negative integers representing the number of points possible on individual assignments. Items corresponds to those in the pointsEarned list.

Returns:

A string containing the letter grade.

Description:

Returns the letter grade corresponding to the overall score computed as the sum of points possible divided by the sum of points earned.

This function should use your getGradeByScore() function to get the final grade string, given the computed overall score.

Example use:

```
>>> strGrade = getGradeByPoints([8,9,15],[10,10,20])
>>> print(strGrade)
B-
```

Function: getGradeRange(strGrade)

Arguments:

strGrade: A string containing the letter grade for which the range is requested.

Returns:

A tuple containing the grade range corresponding to the letter grade. The low end of the grade range should be the first entry and the high end should be the second entry.

Description:

Returns the grade range corresponding to the letter grade.

Example use:

```
>>> tupleGradeRange = getGradeRange('B+')
>>> print(tupleGradeRange)
(87,90)
```

Helpful hints:

How do I get started?

There is a lot to this program. Here are some suggestions:

- Read the problem carefully and write down what steps you need to do for each function.
- Work incrementally.
 - Identify tasks that need to be accomplished to write the functions.
 - Identify needed data types and data structures.
 - Identify necessary methods and constructs needed in the code.
 - Try things in the Python interactive console to see that they work.
 - Write code to do pieces of the problem in order and test the pieces as you go.
 - Print out intermediate results and make sure they are correct before moving on.
- Check carefully at the end to ensure that you have covered everything requested.
 - Check against the assignment details.
 - Test each function thoroughly and carefully.

How do I test the functions?

- Call the functions with a variety of test inputs.
- Check that the functions produce the correct outputs, return values, etc.
- Print the return values.

How will you test the functions for grading?

- Through separate code, I will call your functions with a variety of inputs and test for correct outputs, return values, etc.
- Please ensure that you do not leave active test code in the script you submit. It will interfere with the test process.
- Ensure that you do not leave active test code in the script you submit. It may interfere with the test process.

Upload GradingFunctions.py to Moodle as your submission for this assignment.

EXTENSIONS

You will not submit the answers to these extensions as part of your assignment submission. You should work through and understand them to learn more and practice. You may be held responsible for the content within, so you really should work through them ...

From the Liang book - NOTE: We will **not** use MyProgrammingLab in the book. You should be able to do exercises from Liang without MyProgrammingLab.

1. *Try Exercises 6.1 – 6.5, 6.8 – 6.10, 6.15 and 6.16 in the Programming Exercises Section.*
2. *Experiment with GCDFunction.py (Listing 6.5) and TestGCDFunction.py (Listing 6.6). Recognize how this example shows “modularizing code”.*
3. *Experiment with PrimeNumberFunction.py (Listing 6.7). How does it work? How are the three functions in the script called and used to make work. What is the purpose of the main() function?*
4. *Work through the exercises and examples in Chapter 6. Do the easy ones first. Try some of the more challenging ones.*

From me:

1. Following what was done in our prior Matrix problem (that used lists), write a function that prints the contents of a 5x5 matrix represented by a list of lists as shown and formatted in the prior assignment.
2. Similar to #1 but more general, write a function that prints a two-dimensional matrix represented by a list of lists of arbitrary size m x n.